

Lab 2: Controller Design

MTE 484: Digital Control Systems
Instructor Sergio Livera

Department of Mechatronics and Mechanical Engineering

Levi Rogalla
Alex Veric

November 8, 2026

We acknowledge and promise that:

(a) We are the sole authors of this lab report and associated simulation files/code.

(b) This work represents our original work.

(c) We have not shared detailed analysis or detailed design results, computer code, or Simulink diagrams with any other student.

(d) We have not obtained or looked at lab reports from any other current or former student of ECE 481/ ECE 484 / MTE 484, and we have not let any other student access any part of our lab work.

(e) We have completely and unambiguously acknowledged and referenced all persons and aids used to help us with our work.

- ✓ **Student 1 Name:** Levi Rogalla
- ✓ **Student 1 Signature*:** 
- ✓ **Student 2 Name:** Alex Verić
- ✓ **Student 2 Signature*:** 
- ✓ **Lab Group #:** 57
- ✓ **Station #:** 3

(*) We require an actual signature: a typed name is not an acceptable substitute.

1 Question A

The controller must be designed in a way that allows the closed loop system to display the time-domain characteristics required, which are as follows: the step response steady-state tracking error is zero; he step response 2% settling time is no more than 0.25 seconds; the step response overshoot is no more than 5%; and the motor voltage does not exceed more than 6V in either direction.

The first requirement is already accounted for due to the nature of our plant. According to Internal Model Principle, in order for a closed loop system to have perfect error tracking, the open loop system (which consists of the plant and the controller) must contain a model of the reference signal in the LaPlace domain. In this situation, the given reference signal is a step input, which in the LaPlace domain is represented by $\frac{1}{s}$. The plant being used has already been characterized in the previous lab, and as can be seen in the following equation, already contains a model of the step input. Hence, it is not necessary to consider this in the controller design.

$$P(s) = \frac{K_1}{s(\tau s + 1)} \tag{1}$$

Where $K_1 = -1.8906$ and $\tau = 0.018107$.

The second two requirements, which regard the 2% settling time and the overshoot percentage, must be considered in the design of the controller. These values can be related to the poles of the closed loop system through the following sets of equations, which will determine the desired characteristic polynomial; using this information, as well as the previously derived plant model, the controller can be determined. The following equation shows the relation between the overshoot percentage and the damping ratio, ζ.

$$\zeta = -\frac{ln(OS\%)}{\sqrt{\pi^2 + [ln(OS\%)]}} \tag{2}$$

The following equation shows the relation between the 2% settling time, the damping ratio, ζ, and the natural frequency, ω_n.

$$\omega_n = \frac{4}{\zeta T_{s_{2\%}}} \tag{3}$$

The following equation is the characteristic polynomial of a second order, closed-loop system, in terms of the damping ratio, ζ, and the natural frequency, ω_n.

$$\pi(s) = s^2 + 2\zeta\omega_n s + \omega_n \tag{4}$$

Substituting $T_{s_{2\%}} = 0.25s$ and $OS\% = 0.05$ into the above equations give the following values for the damping ratio, ζ and the natural frequency, ω_n.

$$\zeta = 0.690, \omega_n = 23.185[\frac{1}{sec}] \tag{5}$$

Substituting these values into the characteristic polynomial gives the following equation:

$$\pi(s) = s^2 + 31.995s + 537.542 \tag{6}$$

Using the quadratic formula, as shown below, gives the conjugate pole pair that is required in the closed-loop system to have the required time-domain characteristics:

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \tag{7}$$

Where $a = 1$, $b = 31.995$, $c = 537.542$:

$$s_{1,2} = -15.998 \pm j16.782 \tag{8}$$

The above poles are the conjugate poles which will determine the behaviour of the closed-loop system. However, since the lab requires the controller to be at least first-order (and the derived plant is second order), the characteristic polynomial of the closed-loop system must be third-order. Thus, another pole is required. In order to maintain the desired behaviour already determined, this third pole must be non-dominant - this means it must lie on the real axis and be at least five times further from the imaginary axis than the conjugate pole pair. The reasoning for this is simple: the real part of a pole is what determines the exponential behaviour it contributes to the system. If the real value of the pole is significantly larger (and negative) than the other poles, it will decay much more quickly, resulting in its time-domain behaviour contribution being negligible.

Based on the above description, the third and non-dominant pole of the closed-loop system is $s_3 = -80$.

The third-order characteristic polynomial for the system is as follows:

$$\pi(s) = (s - s_1)(s - s_2)(s - s_3) \tag{9}$$

Substituting the three desired poles, s_1, s_2 and s_3:

$$\pi(s) = (s - (-15.998 + j16.782))(s - (-15.998 - j16.782))(s - (-80)) \tag{10}$$

Expanding and collecting like terms shows the full third order characteristic polynomial and its coefficients, which will be used briefly to determine those of the controller.

$$\pi(s) = s^3 + 111.996s^2 + 3097.252s + 45005.76 \tag{11}$$

The controller design will be done using the Sylvester matrix method, which will be described briefly. First, the coefficients of the characteristic polynomial must be gathered:

$$\alpha_3 = 1, \alpha_2 = 111.996, \alpha_1 = 3097.252, \alpha_0 = 45005.76 \tag{12}$$

Then, the plant must be made into the following form:

$$P(s) = \frac{b_2 s^2 + b_1 s - b_0}{s^2 + a_1 s + a_0} \tag{13}$$

Taking the plant derived in the previous lab, and transposing it to the above form gives the following:

$$P(s) = \frac{0s^2 + 0s - 104.413}{s^2 + 55.227s + 0} \tag{14}$$

And naming the coefficients gives:

$$b_2 = 0, b_1 = 0, b_0 = -104.413 a_1 = 55.227, a_0 = 0 \tag{15}$$

Assuming a proper, first-order controller of the following form:

$$C(s) = \frac{g_1 s + g_0}{f_1 s + f_0} \tag{16}$$

Now, the Sylvester matrix method will be described. It states that a matrix made up of the coefficients of the plant, called the Sylvester matrix, multiplied by an array made up of the coefficients of the controller is equal to an array made up by the coefficients of the resulting characteric polynomial. The following equation displays this:

$$\begin{bmatrix} 1 & 0 & b_2 & 0 \\ a_1 & 1 & b_1 & b_2 \\ a_0 & a_1 & b_0 & b_1 \\ 0 & a_0 & 0 & b_0 \end{bmatrix} \begin{bmatrix} f_1 \\ f_0 \\ g_1 \\ g_0 \end{bmatrix} = \begin{bmatrix} \alpha_3 \\ \alpha_2 \\ \alpha_1 \\ \alpha_0 \end{bmatrix} \tag{17}$$

Substituting all the gathered coefficients from the desired characteristic polynomial and the plant gives:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 55.227 & 1 & 0 & 0 \\ 0 & 55.227 & -104.413 & 0 \\ 0 & 0 & 0 & -104.413 \end{bmatrix} \begin{bmatrix} f_1 \\ f_0 \\ g_1 \\ g_0 \end{bmatrix} = \begin{bmatrix} 1 \\ 111.996 \\ 3097.252 \\ 43005.760 \end{bmatrix} \tag{18}$$

Using simple system of equation rules allows for the calculation of each controller coefficient, as follows:

$$f_1 = 1, f_0 = 56.769, g_1 = 0.363, g_0 = -411.881 \tag{19}$$

Substituting these values into the controller model from equation 16 gives the following:

$$C(s) = \frac{0.363s - 411.881}{s + 56.769} \tag{20}$$

The above controller model was placed into the closed-loop simulation model in MatLab, with the previously derived plant. The reference signal was set to a step function which alternates between $-0.7rad$ and $0.7rad$, with a duty cycle of 50% and a period of 2 seconds, as instructed. Below is the model, followed by the time-domain response.

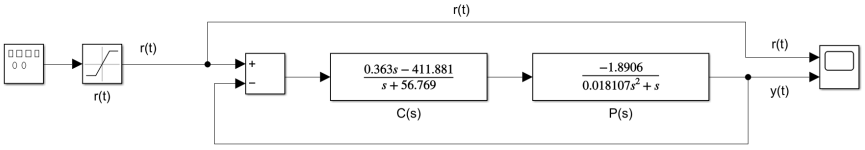


Figure 1: Model of closed-loop system.

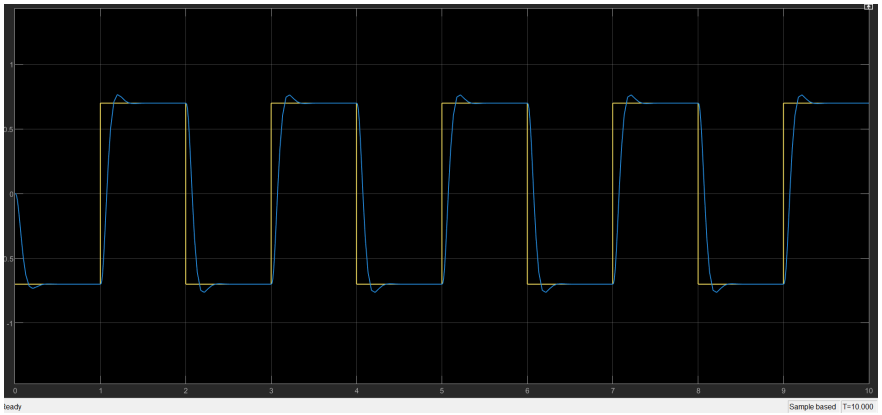


Figure 2: Time-domain response of closed-loop system to reference signal.

From figure 1, it is clear that the steady state error is equal to zero. In order to determine if the controller meets the other two requirements, the peak value and the settling time of the time-domain response signal must be measured.

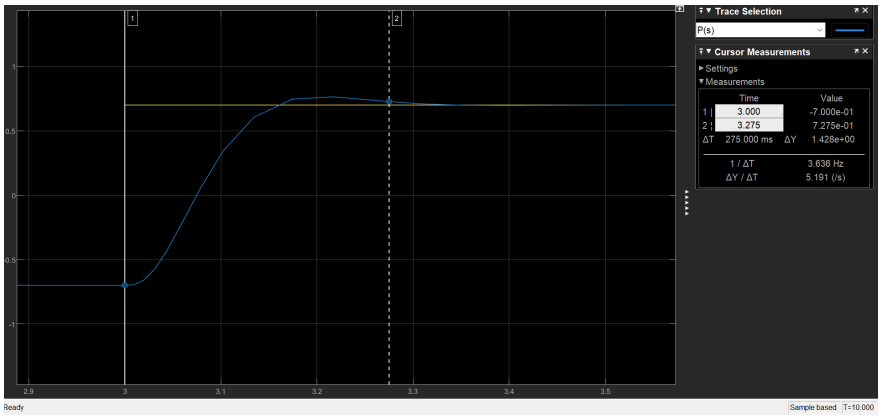


Figure 3: Measurement of 2% settling time

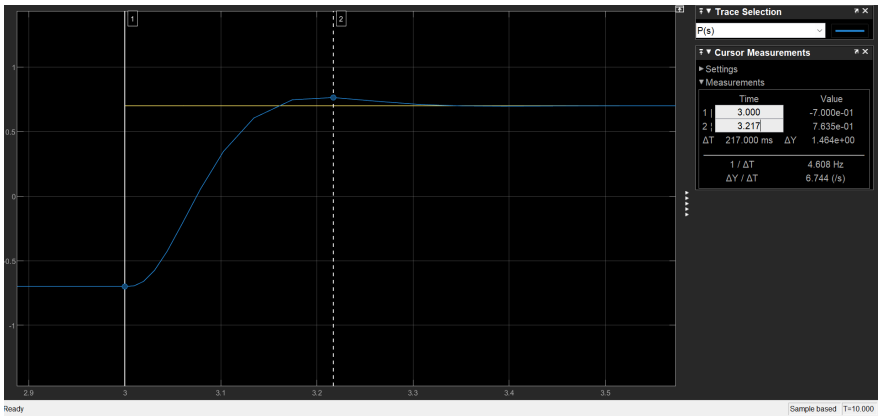


Figure 4: Measurement of overshoot percentage

The 2% settling time is the amount of time after the step response that is required for the

system output to reach 2% of the reference signal. Since the reference signal goes from -0.7 to 0.7, the total value of the step function is 1.4. Therefore, the 2% settling time occurs when the signal reaches 102% of 1.4, which is 1.428. The measurement in figure 1 shows that this occurs 0.275s after the unit step, which is slower than the required 0.25s.

The overshoot percentage, which is required to be no more than 5%, is measured in figure 1. The peak difference is measured to be 1.464; 105% of the unit step value would be 1.47, and so the overshoot percentage of this controller is within the acceptable range.

Since the 2% settling time is dependent on the natural frequency, ω_n, this would need to be modified in order to prevent the settling time from going above its limit. However, this would be time consuming, as it would require redoing nearly the entire process as described above. `systune`, a tool in MatLab, is a much more efficient way of tuning the existing controller to fit within the required settling time. The newly tuned controller has the following values, and its response to a step input can be seen in figure 1.

$$C(s) = \frac{-2.452s - 135.4}{s + 24.95} \tag{21}$$

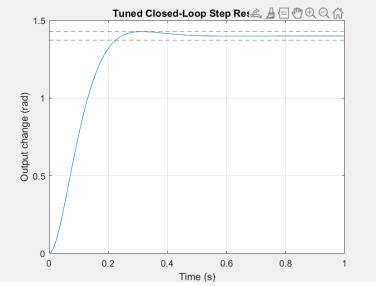


Figure 5: Tuned controller performance to step input.

The tuned controller has a 2% settling time of $0.2252s$ and an overshoot percentage of exactly 2.00%, both of which fit the required time-domain performance. The new controller is also shown to have a voltage within $\pm 6V$, as can be seen by the signal output by C(s) in figure 1.

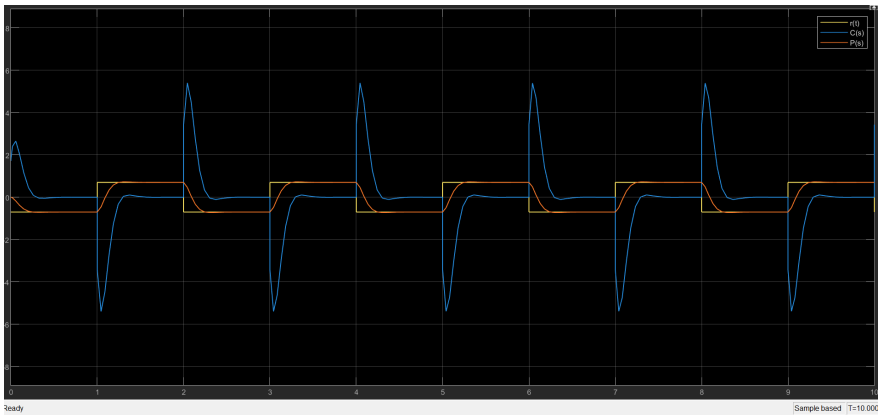


Figure 6: Motor voltage shown to be within $\pm 6V$ when using the tuned controller.

2 Question B

The discretization of a controller is necessary to allow a microcontroller, such as the Arduino Mega, to run the controller digitally. Software controllers cannot run in continuous time, and so have to sample the continuous time values, therefore making the system discrete. While a continuous controller can use derivatives to transform between LaPlace and time domains, discrete systems are non-continuous and therefore can not utilize derivatives. This results in discretization being necessary, and this can be done using one of two methods: bilinear transformation or step-invariant transformation. Bilinear transformation guarantees the stability of an already stable continuous controller, and is better at maintaining the time-domain behaviour of a continuous controller, while step-invariant transformation does not guarantee stability, but better maintains frequency domain responses. Since time-domain characteristics are those which are required, and stability is paramount in this system, bilinear transformation will be used to discretize the continuous controller.

Bilinear transformation involves replacing the Laplace domain variable s with the discrete domain variable z, using the following equation:

$$s = \frac{2}{T}\frac{z-1}{z+1} \tag{22}$$

Where T is the sampling period, which has been chosen to be 0.0005, making the sampling frequency equal to $2kHz$. Analyzing the time-domain step response, there are a few periods in which fast response is important, such as the peak time and the settling time, which are required characteristics that must be maintained during discretization. At these points, the controller must react quickly to limit the overshoot around the peak time and to reduce oscillations around the 2% settling time. Since the time constant which governs these values is 0.0625, the sampling time period was chosen to be roughly one hundred times smaller to allow the controller to react appropriately.

(Note: 200 Hz was initially used for the sampling frequency, but it was experimentally determined that this was too slow of a response to obtain the desired time-domain characteristics; for this reason, the frequency was increased by a factor of ten.)

Substituting s using equation 22 into the equation 16 for the continuous controller gives a new, discrete controller formula that should replicate its time-domain behaviour:

$$C[z] = \frac{(2g_1 + Tg_0)z + (Tg_0 - 2g_1)}{(2f_1 + Tf_0)z + (Tf_0 - 2f_1)} \tag{23}$$

Substituting the new coefficients of the tuned controller, as seen in equation 21, as well as the chosen sampling period, T, gives the following:

$$C[z] = \frac{-4.9717z + 4.8363}{2.012475z - 1.987525} \tag{24}$$

The discrete controller was then input into MatLab, where it was compared with the continuous controller step response.

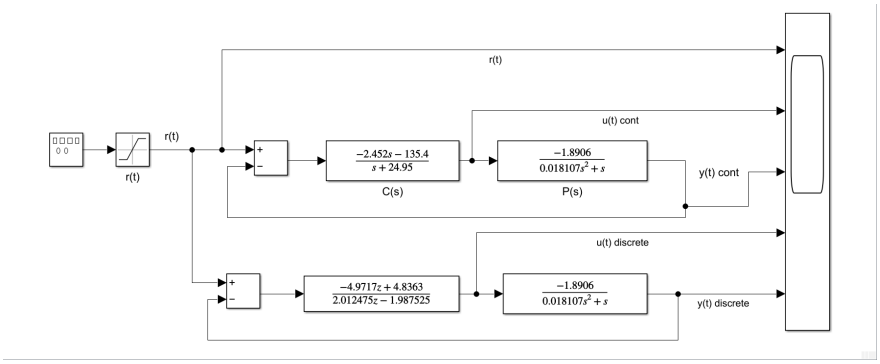


Figure 7: Discrete controller model in MatLab

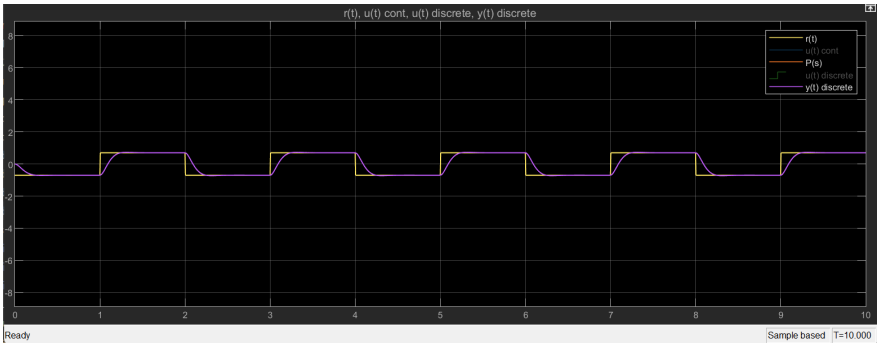


Figure 8: Discrete and continuous models step response compared. As can be seen, there is little to no visible difference between the time-domain characteristics of the two models.

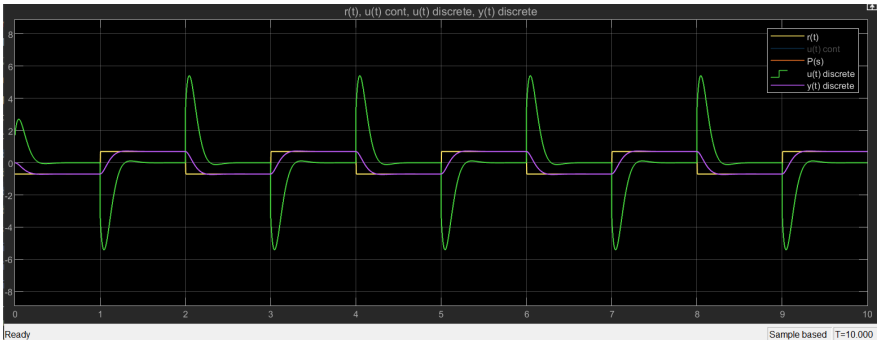


Figure 9: Motor voltage when using the discrete controller; it does not encroach $\pm 6V$

3 Question C

Now that the controller has been discretized, it can now be used to run the control system on the Arduino, which works digitally and in fixed sample intervals. However, there is one last step before the Arduino can use the controller: it must be transformed from the Z-domain into the sampling domain, which, unlike its counterpart, the time-domain, computes in fixed time intervals rather than continuous time. In order to do this, the controller must first be represented in terms of the signals which act as its input and output, as seen below:

$$C[z] = \frac{U[z]}{E[z]} = \frac{-4.9717z + 4.8363}{2.012475z - 1.987525} \tag{25}$$

Where U[z] is the discrete output of the controller, and E[z] is the discrete input. This equation must be rearranged as follows, to obtain a value for the output, based on the input:

$$U[z] = \frac{-4.9717z + 4.8363}{2.012475z - 1.987525} E[k] \tag{26}$$

Applying the inverse Z-transform:

$$u[k] = 0.987602u[k-1] - 2.470441e[k] + 2.403160e[k-1]. \tag{27}$$

Note, this can also be written as follows:

$$u[k] = \frac{-4.9717}{2.012475} q[k] + \frac{4.8363}{2.012475} q[k-1] \tag{28}$$

$$q[k] = e[k] - \frac{-1.987525}{2.012475} q[k-1] \tag{29}$$

where $q[k]$ represents the internal state of the system.

Where k is an arbitrary sample, similar to how t is an arbitrary continuous unit of time, in the time-domain. The above equation was implemented in the Arduino code, along with its appropriate sampling time of $0.0005s$. The entire system was then tested with the a square wave input signal at 50% duty cycle and with a period of $2s$. The system time response is shown below in figure 3.

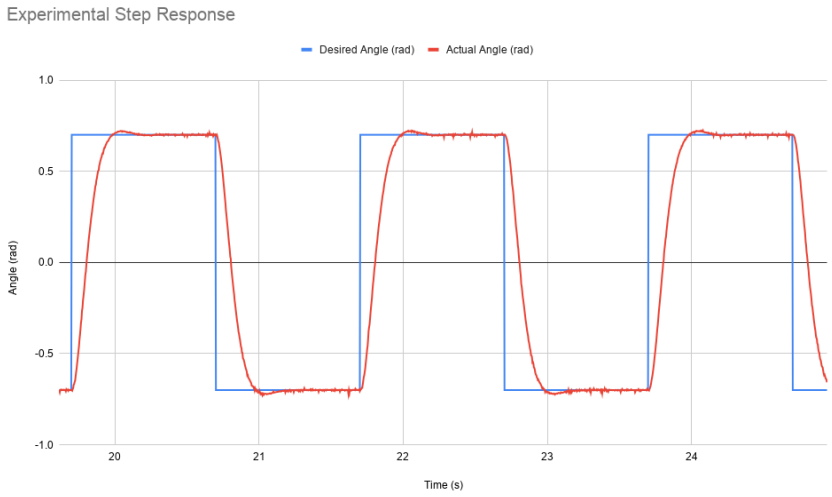


Figure 10: Step response of the motor

In order to ensure the time-domain characteristics required for this system are reached, figure 3 must be further inspected. The 2% settling time is reached almost exactly $0.25s$ after the step input is triggered. Additionally, the peak of the motor position is $0.72087rad$, which is roughly 2% of the unit step value; this meets the required maximum overshoot percentage of 5%. See below the exact values of the first recorded time-domain response from figure 3:

$$
\begin{aligned}
2\%\text{ range} &= (0.672, 0.728),\\
T_{\text{signal}} &= 19.7017315\,\text{s},\\
T_{s,\,2\%} &= 19.951723\,\text{s},\\
T_{s,\,2\%} - T_{\text{signal}} &= 0.2499915\,\text{s}.
\end{aligned}
$$

Using the following equations to obtain $OS\%$:

$$
OS\% = \frac{u(t)_{max} - r(t)_{max}}{|r(t)|} \times 100\% \tag{30}
$$

The following OS% can be obtained:

$$
OS\% = 1.43928\% \tag{31}
$$

The figure below shows the motor voltage as well, which never exceeds $\pm 6V$:

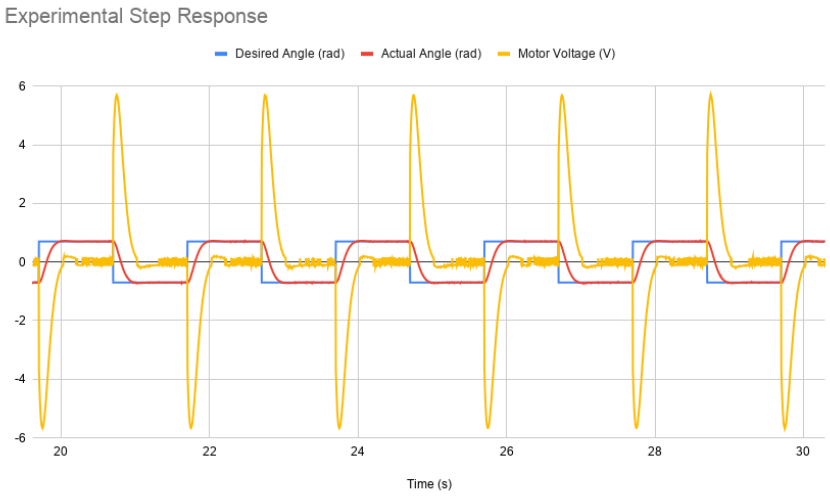


Figure 11: Step response and voltage of motor.

There are several differences between the performance of the simulated system and the real system when using the discretized controller. The 2% settling time of the real system is slightly longer than the simulated one, which can likely be attributed to the time delay introduced by the controller running on real hardware rather than inside a simulation. This presents itself in the form of a very slight delay between the unit step and the beginning of the motor movement, as can be seen in figure 3, which doesn't exist in the simulated system, in figure 1

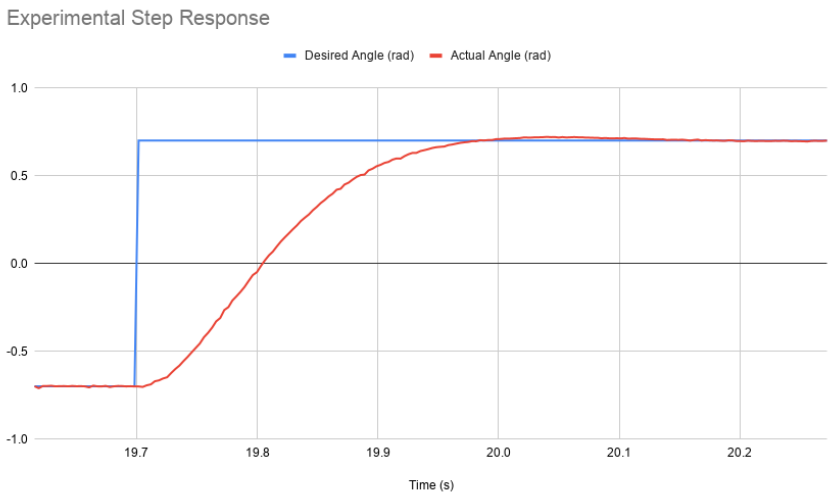


Figure 12: Zoomed in chart of the motor step response.

The experimental overshoot percentage is smaller than the simulated system. This was expected, as the forces introduced in a real system, such as internal friction or air resistance, increase the damping ratio of the system, resulting a smaller overshoot. The motor voltage of the system was seen to be higher than that of the simulated system, which was, once again, expected behaviour. The constant voltage added to the motor voltage by the Arduino code was one culprit of this higher motor voltage, as was the controller having to add more voltage as a result of the aforementioned internal friction and air resistance which do not exist in the simulated system.

4 Question D

The stiction values from Lab 1 were found to no longer hold up when the system had increased weight due to the beam now being attached. The new stiction values were found in the same way as before: a program was used to increase the motor voltage from zero in very small increments until the motor began to move - this was done in both directions. The values were changed for both directions, since the beam being attached to the motor gear changes the voltage required for both directions. In the clockwise direction, the beam pushes in the direction of the motor movement, which means the controller does not need to accout for as much stiction as before. In the counter-clockwise direction, the beam pushes in the opposite direction of the motor movement, meaning the controller must use a higher voltage than before to overcome the stiction. While running this program, it was imperative that the ball was on the beam, as this was another factor that would contribute weight to the system, and so the stiction values needed to account for this.

As can be seen in figure 4, unless the motor voltage is set to zero, the output of the controller always causes the motor to move; this is the succesful result of finding the correct stiction values and accomodating for them in code.

The values for stiction were found to be $-0.5763V$ in the counter-clockwise direction, and $-0.1034V$ in the clockwise position.

The emergency stop function, which stops the motor function at a certain motor position

($\pm 0.8 rad$) to stop potential damage to the system, was also implemented in the code. As can be seen in figure 4, the emergency stop works as intended.

5 Question E

Now that the inner control loop, which transforms the desired motor angle values into accurate motor movement, is almost complete, it is time to start working towards the outer control loop, which must determine the motor angle required in order to move the ball towards a specific position on the beam. A required step in this process is to have an effective method of determining the position of the ball on the beam; this will be used in the outer loop's error measurements. The beam has a built in sensor, whose output represents the current position of the ball; however, the sensor must be characterized in order to determine how the sensor value corresponds to the position of the ball on the beam in meters.

This can be done using the following formula as a conversion between sensor output and ball position:

$$p_{meters} = m \times p_{sensor} + b \tag{32}$$

Where the m and b values must be determined through a system of equations, using experimental values of p_{meters} and p_{sensor} to do so.

The position of the ball on the beam was manually changed to two different points: one at each the furthermost left and right side of the beam. These two points are sufficient to determine a linear relationship. Using the measurement convention given in the lab manual, it must be noted that the ball will never reach $0m$ or Lm, where L is the length of the beam; this is due to the small stoppers that rest at each end of the beam to ensure the ball doesn't fall off. At each point, the distance from the beam origin was measured, and the value printed into the serial monitor was also noted, providing values of p_{meters} in meters and p_{sensor} in ADC counts. It should also be mentioned that each ADC "value" recorded in this report is actually the average of multiple ADC values at that specific ball position, effectively reducing the effect of ADC and sensor noise.

The following values were noted:

$$p_{meters_0} = 0.030m, p_{sensor_0} = 2322.44 p_{meters_1} = 0.389m, p_{sensor_1} = 35333.44$$

The following system of equations was then made from the above values:

$$\begin{bmatrix} 0.030 \\ 0.389 \end{bmatrix} = \begin{bmatrix} 2322.44 & 1 \\ 35333.44 & 1 \end{bmatrix} \begin{bmatrix} m \\ b \end{bmatrix} \tag{33}$$

The above system of equations was solved for m and b resulting in the following values:

$$m = 1.0875 \times 10^{-5}, b = 4.7431 \times 10^{-3} \tag{34}$$

After implemented equation 32 into the Arduino code, using the values provided from equation 34, the ball position could now be determined using the beam sensor output.

6 Question F

Now that the ball position can reliably be determined, it is now necessary to determine the relation between the input and the output of the outer-loop controller, which will be the ball location and the motor gear angle, respectively. This transfer function has two parts: the first is the transfer function between the motor gear angle and the beam angle. This has already been determined in the lab manual to be the equation 35.

$$\frac{\Phi(s)}{\Theta(s)} = 0.0609 \tag{35}$$

Where $\Phi(s)$ represents the beam angle, and $\Theta(s)$ represents the motor angle. The second part is the relation between the beam angle and the ball position, which takes the form of equation 36.

$$\frac{Y(s)}{\Phi(s)} = \frac{K_3}{s^2} \tag{36}$$

Where $Y(s)$ represents the position of the ball on the beam. Rearranging equation 36 and applying the inverse Laplace transform provides the following equation:

$$\ddot{y}(t) = K_3\phi(t) \tag{37}$$

Which shows that $ddoty(t)$, which represents the second derivative of position, also known as acceleration, is dependent on the beam angle. The knowledge of the basic kinematics equation which defines acceleration is shown below:

$$y(t) = y_0 + doty_0(t - t_0) + \frac{1}{2}ddoty(t - t_0)^2 \tag{38}$$

Substituting equation 37 into equation 38 heeds the following equation:

$$y(t) = y_0 + doty_0(t - t_0) + \frac{1}{2}K_3\phi(t - t_0)^2 \tag{39}$$

Isolating for K_3 gives the following:

$$K_3 = 2\frac{y - y_0 - doty_0(t - t_0)}{\phi(t - t_0)^2} \tag{40}$$

Where y is the current position of the ball, y_0 is the initial position of the ball, $doty_0$ is the initial speed of the ball, ϕ is the beam angle for the given experiment, t_0 is the initial time the ball is placed and t is the current time at which y is being measured.

The experiments were then performed to determine K_3. At a given beam angle and initial time, the ball would be placed at a given initial position, at the initial speed of zero. At any arbitrary point of time in the drop, the position and its time can be inputted into the equation, allowing for the value of K_3 to be solved. This experiment was performed six times, at three different positive beam angles and three different negative angles, to ensure that the value of K_3 was consistent across all different beam angles. The following beam angles were chosen, as they were determined to be representative of those used while controlling the ball position. The inner-loop controller was used to fix the beam at a specific angle, allowing for the experiment to take place.

$$\begin{aligned} &\phi_0 = 0.2rad, \quad &&\phi_1 = -0.2rad, \quad &&\phi_2 = 0.3rad, \\ &\phi_3 = -0.3rad, \quad &&\phi_4 = 0.4rad, \quad &&\phi_5 = -0.4rad \end{aligned}$$

The figure below shows the entire experiment process, including all six different beam angles.

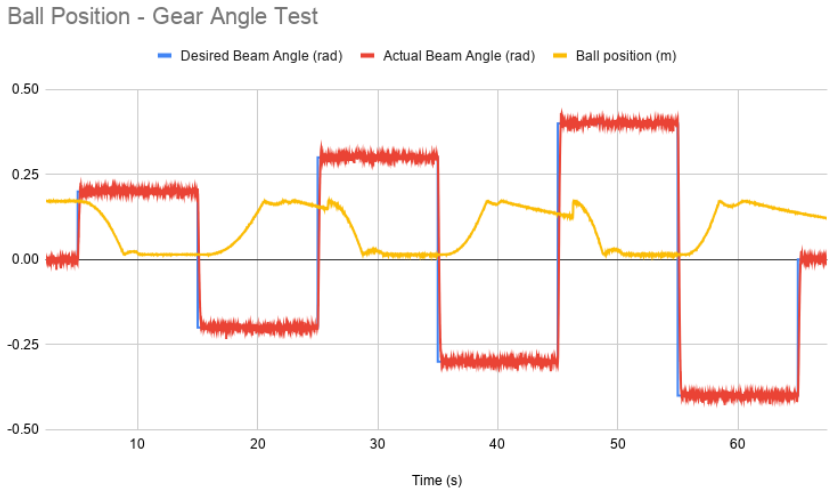


Figure 13: Experimental determination of K_3

From the above data, the values for y, y_0, $doty_0$, t_0, t and ϕ can be recorded for each experiment, as is shown in the table below:

Table 1: Ball position - Gear Angle test

Beam angle ϕ [rad]	**Current ball position y [m]**	**Initial ball position y_0 [m]**	**Initial ball speed $\dot{y}_0$ [m/s]**	**Current time t [s]**	**Initial time t_0 [s]**
0.2	0.11829066	0.17151371	0	7.321955	5.187743
-0.2	0.0964533	0.01474824	0	19.195892	15.310882
0.3	0.05731362	0.16755515	0	28.310871	26.289441
-0.3	0.14961113	0.01784766	0	38.823754	36.122445
0.4	0.06996144	0.15256917	0	48.298498	47.134454
-0.4	0.14098713	0.01495486	0	58.162953	55.559953

Substituting the above values into equation 40 provides the following values for K_3:

Table 2: Experimental values of K_3

Beam angle ϕ [rad]	**K_3 constant**
0.2	-0.11684890
-0.2	-0.05413332
0.3	-0.17986082
-0.3	-0.12038025
0.4	-0.30482592
-0.4	-0.09300439

Averaging out the values gives a final result for K_3:

$$K_{3_{avg}} = -0.14484227$$

References

1. S. Livera, *MTE 484: Digital Control Systems — Lab 2*, Fall 2026.

7 Appendices

7.1 Appendix A: Code

Listing 1: main.ino

```
#include "api/Common.h"
#include "src/geeWhiz/geeWhiz.h"
#include "src/motor.h"
#include "src/position.h"
#include "src/transfer.h"
#include <Arduino.h>
#include <cstdint>

#define ISR_PERIOD_US 500
// ================== Pins ==================
int MOT_PIN = A0; // motor angle sensor
int BAL_PIN = A1; // ball position sensor

int PWM_PIN = D9; // motor PWM
int DIR_PIN = D8; // direction

typedef enum {
  APP_STATE_CALCULATE_OVERSHOOT,
  APP_STATE_CALCULATE_STICKTION,
  APP_STATE_SINE_WAVE,
  APP_STATE_SQUARE_WAVE,
  APP_STATE_CHARACTERIZE_BALL_SENSOR,
  APP_STATE_GRAVITY_THING,
  APP_STATE_ESTOP,
} AppState;

static AppState s_app_state;
uint32_t logging_start_us;

// ================== Setup ==================
void setup()
{

  analogReadResolution(14);
  pinMode(A5, OUTPUT); // A5 can be used to measure cycle time using an
      oscilloscope by connecting the scope to the Arduino Box Motor
     Leads
  Serial.begin(115200);
```

```
  pot_angle_config(MOT_PIN, 3);
  sense_distance_config(BAL_PIN, 3);
  motor_config(PWM_PIN, DIR_PIN);

  delay(300);

  pot_angle_populate();
  motor_init();
  set_control_interval_us(ISR_PERIOD_US); // 2 kHz

  s_app_state = APP_STATE_GRAVITY_THING;
  logging_start_us = micros();
  Serial.println("initialization complete!");
}

// ================== Loop ==================
uint64_t sample = 0;
void loop()
{
  switch (s_app_state)
  {
  case APP_STATE_CALCULATE_OVERSHOOT:
    motor_controller_enable(true);
    calculate_overshoot(1000);
    break;
  case APP_STATE_CALCULATE_STICKTION:
    motor_controller_enable(false);
    calculate_stiction();
    break;
  case APP_STATE_SINE_WAVE:
    motor_controller_enable(true);
    theta_desired_set(0.7 * calculate_sine_wave(8000, 0.0f));
    break;
  case APP_STATE_SQUARE_WAVE:
    motor_controller_enable(true);
    theta_desired_set(0.7 * calculate_square_wave(2000, 0.5, 0.0f));
    break;
  case APP_STATE_CHARACTERIZE_BALL_SENSOR: {
    motor_controller_enable(true);
    float theta =
        0.3 * (0.5 * (calculate_square_wave(13 * 1000, 0.1, 0.0f) + 1)
           -
               0.5 * (calculate_square_wave(13 * 1000, 0.1, 180.0f) +
                  1));
    theta_desired_set(theta);
    break;
  }
```

```
  case APP_STATE_GRAVITY_THING: {
    motor_controller_enable(true);

    uint32_t elapsed_us = micros() - logging_start_us;
    if (elapsed_us < 5e6) {
      theta_desired_set(0.0);
    } else if (elapsed_us < 15e6) {
      theta_desired_set(0.2);
    } else if (elapsed_us < 25e6) {
      theta_desired_set(-0.2);
    } else if (elapsed_us < 35e6) {
      theta_desired_set(0.3);
    } else if (elapsed_us < 45e6) {
      theta_desired_set(-0.3);
    } else if (elapsed_us < 55e6) {
      theta_desired_set(0.4);
    } else if (elapsed_us < 65e6) {
      theta_desired_set(-0.4);
    } else {
      theta_desired_set(0.0);
    }

    break;
  }
  case APP_STATE_ESTOP:
    motor_controller_enable(false);
    theta_desired_set(0.7 * calculate_square_wave(2000, 0.5, 0.0f));
    Serial.println("Estopped");
    delay(200);
    break;
  }

  float angle = pot_angle_read_eng();
  Serial.print(sample);
  Serial.print(",");
  Serial.print(micros() - logging_start_us);
  Serial.print(",");
  Serial.print(theta_desired_get(), 5);
  Serial.print(",");
  Serial.print(motor_voltage_get(), 5);
  Serial.print(",");
  Serial.print(angle, 5);
  Serial.print(",");
  Serial.println(sense_distance_read_eng(), 8);
  Serial.println(pot_angle_read_raw());
}

// ================ Control ISR ================
```

```
static float controller_init_state = 0;
void interval_control_code(void)
{
  sample++;
  pot_angle_populate();
  sense_distance_populate();
  float angle = pot_angle_read_eng();

  if (abs(angle) > 0.8)
  {
    motor_voltage_no_stick_set(0);
    motor_controller_enable(false);
    s_app_state = APP_STATE_ESTOP;
  }

  if (motor_controller_enabled()) {
    float theta_desired = theta_desired_saturated_get();
    motor_voltage_no_stick_set(motor_controller_theta_to_volt_1(
        theta_desired, angle, &controller_init_state));
  }
}
```

Listing 2: motor.h

```
#ifndef MOTOR_H
#define MOTOR_H

#include <Arduino.h>

void motor_config(int pwm_pin, int dir_pin);

int motor_init();

void motor_voltage_no_stick_set(float volts);

float motor_voltage_get();

void theta_desired_set(float theta);

float theta_desired_get(void);

float theta_desired_saturated_get(void);

void calculate_stiction(void);

void calculate_overshoot(uint32_t period_ms);

float calculate_sine_wave(uint32_t period_ms, float
```

```
    phase_offset_degrees);

// duty_cycle is the fraction of each period spent at positive
    amplitude (0.0 to 1.0).
float calculate_square_wave(uint32_t period_ms, float duty_cycle,
                            float phase_offset_degrees);

#endif
```

Listing 3: motor.cpp

```
#include "Arduino.h"
#include "geeWhiz/geeWhiz.h"
#include "position.h"
#include "transfer.h"
#include <atomic>
#include <cmath>
#include <pwm.h>

#define POS_V_STICTION -0.1034f
#define NEG_V_STICTION -0.5763f
#define THETA_DESIRED_MAX 0.7f
#define THETA_SWEEP_AMPLITUDE 1

static int s_motor_dir_pin;
static bool s_motor_dir_pin_configured = false;
static PwmOut s_motor_pwm(0);
static bool s_motor_pwm_configured = false;
static constexpr float PWM_HZ = 24000.0f;
static constexpr float VMAX = 6.0f;
static constexpr float DUTY_MAX = 99.2f;
static std::atomic<float> s_theta_desired{0.0f};
static float s_motor_last_set_voltage = 0;

// must be called before motor init
void motor_config(int pwm_pin, int dir_pin)
{
  PwmOut configured_pwm(pwm_pin);
  s_motor_pwm = configured_pwm;
  s_motor_pwm_configured = true;

  s_motor_dir_pin = dir_pin;
  s_motor_dir_pin_configured = true;
}

int motor_init()
{
  if (!s_motor_pwm_configured || !s_motor_dir_pin_configured)
```

```
  {
    return -1;
  }

  pinMode(s_motor_dir_pin, OUTPUT);
  digitalWrite(s_motor_dir_pin, LOW);

  s_motor_pwm.begin(PWM_HZ, 0.0f);
  return 0;
}

static void motor_voltage_set(float volts)
{
  if (volts > VMAX)
    volts = VMAX;
  if (volts < -VMAX)
    volts = -VMAX;

  digitalWrite(s_motor_dir_pin, (volts >= 0.0f) ? HIGH : LOW);
  float duty = (fabsf(volts) / VMAX) * 100.0f;
  if (duty > DUTY_MAX)
    duty = DUTY_MAX;

  s_motor_pwm.pulse_perc(duty);
}

void motor_voltage_no_stick_set(float volts) {
  if (volts > 0.0f)
  {
    s_motor_last_set_voltage = volts + POS_V_STICTION;
  }
  else if (volts < 0.0f)
  {
    s_motor_last_set_voltage = volts + NEG_V_STICTION;
  }
  else
  {
    s_motor_last_set_voltage = 0;
  }
  motor_voltage_set(s_motor_last_set_voltage);
}

float motor_voltage_get() { return s_motor_last_set_voltage; }

static float theta_desired_saturate(float theta_desired)
{
  if (theta_desired > THETA_DESIRED_MAX)
  {
```

```
    return THETA_DESIRED_MAX;
  }
  else if (theta_desired < -THETA_DESIRED_MAX)
  {
    return -THETA_DESIRED_MAX;
  }
  else
  {
    return theta_desired;
  }
}

void theta_desired_set(float theta)
{
  s_theta_desired.store(theta, std::memory_order_relaxed);
}

float theta_desired_get(void)
{
  return s_theta_desired.load(std::memory_order_relaxed);
}

float theta_desired_saturated_get(void)
{
  return theta_desired_saturate(theta_desired_get());
}

void calculate_overshoot(uint32_t period_ms)
{
  float theta0 = 0.0f;
  float theta1 = 0.2f;

  static uint32_t t0 = millis();
  static bool state = false;

  if (millis() - t0 > period_ms)
  {
    state = !state;
    t0 = millis();
  }

  float theta_target = state ? theta1 : theta0;
  theta_desired_set(theta_target);
}

float calculate_sine_wave(uint32_t period_ms, float
   phase_offset_degrees) {
  if (period_ms == 0)
```

```
  {
    return 0.0f;
  }

  float cycle_fraction = static_cast<float>(millis() % period_ms) /
                         static_cast<float>(period_ms) +
                         phase_offset_degrees / 360.0f;
  float phase = 2.0f * PI * cycle_fraction;
  return THETA_SWEEP_AMPLITUDE * sinf(phase);
}

float calculate_square_wave(uint32_t period_ms, float duty_cycle,
                            float phase_offset_degrees) {
  if (period_ms == 0)
  {
    return 0.0f;
  }

  float cycle_fraction = static_cast<float>(millis() % period_ms) /
                         static_cast<float>(period_ms) +
                         phase_offset_degrees / 360.0f;
  cycle_fraction -= floorf(cycle_fraction);
  return cycle_fraction < duty_cycle ? THETA_SWEEP_AMPLITUDE
                                     : -THETA_SWEEP_AMPLITUDE;
}

/// @brief Calculate and print stiction of motor, disables interrupts
void calculate_stiction(void)
{
  const float voltage_step = 0.001;
  const float theta_threshold = 0.02;

  float theta;
  float motor_voltage = 0.0;

  // Bring motor to theta = 0 using the normal controller.
  Serial.println("Setting theta to 0 using closed loop control...");
  theta_desired_set(0.0);
  motor_controller_enable(true);
  delay(2000);
  motor_controller_enable(false);

  // Find stiction in both directions.
  float stiction_vals[2] = {0, 0};

  for (int direction = -1; direction <= 1; direction += 2) {
    motor_voltage = motor_voltage_get();
    bool moved = false;
```

```
    float theta_init = pot_angle_read_eng();

    while (!moved) {
      for (int t = 30; t > 0; t--) {
        theta = pot_angle_read_eng();

        motor_voltage_set(motor_voltage);

        if (abs(theta - theta_init) >= theta_threshold) {
          moved = true;
          break;
        }

        Serial.print(motor_voltage, 8);
        Serial.print(",");
        Serial.println(theta, 5);

        delay(4);
      }

      motor_voltage += direction * voltage_step;
    }

    Serial.println("done");

    stiction_vals[(direction + 1) / 2] = motor_voltage;

    // Return to zero using the normal controller.
    motor_controller_enable(true);
    delay(2000);
    motor_controller_enable(false);
  }

  motor_controller_enable(true);
  theta_desired_set(0.0);

  Serial.print("negative voltage stiction:");
  Serial.println(stiction_vals[0], 8);
  Serial.print("positive voltage stiction:");
  Serial.println(stiction_vals[1], 8);

  Serial.println("stiction calculation complete... delaying for 10
     seconds");
  delay(1000 * 60 * 10);

  motor_controller_enable(false);
}
```

Listing 4: position.h

```
#ifndef POSITION_H
#define POSITION_H

void pot_angle_config(int angle_pin, int samples);

void pot_angle_populate(void);

int pot_angle_read_raw(void);

float pot_angle_read_eng(void);

void sense_distance_config(int distance_pin, int samples);

void sense_distance_populate(void);

int sense_distance_read_raw(void);

float sense_distance_read_eng(void);

#endif
```

Listing 5: position.cpp

```
#include "position.h"
#include "transfer.h"
#include <analog.h>
#include <atomic>

static int s_angle_pin = 0;
static int s_angle_samples = 1;
static std::atomic<int> s_angle_raw{0};

static int s_position_pin = 0;
static int s_position_samples = 1;
static std::atomic<int> s_position_raw{0};

static void sensor_populate(int pin, int samples, std::atomic<int> *
    output) {
  int sum = 0;
  for (int i = 0; i < samples; i++) {
    sum += analogRead(pin);
  }

  // round to nearest integer when averaging
  int raw = (sum + samples / 2) / samples;
  output->store(raw, std::memory_order_relaxed);
}
```

```
void pot_angle_config(int angle_pin, int samples) {
  s_angle_pin = angle_pin;
  s_angle_samples = samples > 0 ? samples : 1;
}

void pot_angle_populate(void) {
  sensor_populate(s_angle_pin, s_angle_samples, &s_angle_raw);
}

int pot_angle_read_raw(void) {
  return s_angle_raw.load(std::memory_order_relaxed);
}

float pot_angle_read_eng(void) {
  return pot_angle_raw_to_rad(pot_angle_read_raw());
}

void sense_distance_config(int position_pin, int samples) {
  s_position_pin = position_pin;
  s_position_samples = samples > 0 ? samples : 1;
}

void sense_distance_populate(void) {
  sensor_populate(s_position_pin, s_position_samples, &s_position_raw);
}

int sense_distance_read_raw(void) {
  return s_position_raw.load(std::memory_order_relaxed);
}

float sense_distance_read_eng(void) {
  return sense_distance_raw_to_m(sense_distance_read_raw());
}
```

Listing 6: transfer.h

```
#ifndef TRANSFER_H
#define TRANSFER_H

#define MOTOR_CONTROLLER_KP -28

float pot_angle_raw_to_rad(int v);

float sense_distance_raw_to_m(int v);

float motor_controller_theta_to_volt_0(float theta_target, float theta)
    ;
```

```
float motor_controller_theta_to_volt_1(float theta_target, float theta,
                                       float *previous_state);

void motor_controller_enable(bool enable);

bool motor_controller_enabled();

#endif
```

Listing 7: transfer.cpp

```
#include "transfer.h"

#define ANGLE_SLOPE -3.66300473e-04f
#define ANGLE_INTERCEPT 1.79615437f

#define DISTANCE_SLOPE 1.0875162824513041e-05f
#define DISTANCE_INTERCEPT 0.004743086849837932

#define MOTOR_CONTROLLER_FIRST_ORDER_F0 -1.987525f
#define MOTOR_CONTROLLER_FIRST_ORDER_F1 2.012475f
#define MOTOR_CONTROLLER_FIRST_ORDER_G0 4.8363f
#define MOTOR_CONTROLLER_FIRST_ORDER_G1 -4.9717f

float pot_angle_raw_to_rad(int v) {
  return (float)v * ANGLE_SLOPE + ANGLE_INTERCEPT;
}

float sense_distance_raw_to_m(int v) {
  return (float)v * DISTANCE_SLOPE + DISTANCE_INTERCEPT;
}

float motor_controller_theta_to_volt_0(float theta_target, float theta)
    {
  float err = theta_target - theta;
  return err * MOTOR_CONTROLLER_KP;
}

float motor_controller_theta_to_volt_1(float theta_target, float theta,
                                       float *previous_state) {
  float err = theta_target - theta;
  float state = err - (MOTOR_CONTROLLER_FIRST_ORDER_F0 /
                       MOTOR_CONTROLLER_FIRST_ORDER_F1) *
                          *previous_state;
  float u =
      (MOTOR_CONTROLLER_FIRST_ORDER_G1 /
          MOTOR_CONTROLLER_FIRST_ORDER_F1) *
```

```
          state +
      (MOTOR_CONTROLLER_FIRST_ORDER_G0 /
         MOTOR_CONTROLLER_FIRST_ORDER_F1) *
          *previous_state;
  *previous_state = state;
  return u;
}

static bool s_enable_control_isr = false;
void motor_controller_enable(bool enable) { s_enable_control_isr =
   enable; }
bool motor_controller_enabled() { return s_enable_control_isr; }
```

Listing 8: geeWhiz.h

```
#pragma once
#include <Arduino.h>

// Start a periodic ISR at 'interval_us' using a GPT timer (no PWM
   conflict)
void set_control_interval_us(uint32_t interval_us);

// Your sketch must define ONE of these names (either is accepted)
extern "C" void interval_control_code(void) __attribute__((weak));
extern "C" void intervalControlCode(void) __attribute__((weak));
```

Listing 9: geeWhiz.cpp

```
#include "geeWhiz.h"
#include <FspTimer.h> // UNO R4 timer helper

static FspTimer controlTimer; // separate GPT for the control ISR

// ---- ISR adapter ----
static void timer_cb_adapter(timer_callback_args_t *)
{
  if (interval_control_code)
  {
    interval_control_code();
    return;
  }
  if (intervalControlCode)
  {
    intervalControlCode();
    return;
  }
  // neither defined -> do nothing
}
```

```
// ---- API ----
void set_control_interval_us(uint32_t interval_us)
{
  if (interval_us == 0)
    interval_us = 1000;
  const float freq_hz = 1000000.0f / interval_us;

  uint8_t timer_type = GPT_TIMER;
  int8_t tindex = FspTimer::get_available_timer(timer_type); // prefer
     non-PWM GPT

  bool forced = false;
  if (tindex < 0)
  {
    // As a fallback, allow using a PWM-reserved GPT if needed
    tindex = FspTimer::get_available_timer(timer_type, true);
    forced = (tindex >= 0);
  }
  if (tindex < 0)
    return; // no timer available

  if (forced)
  {
    // Permit using a PWM-reserved GPT for the timer.
    FspTimer::force_use_of_pwm_reserved_timer();
  }

  // Begin periodic timer -> IRQ -> open -> start (open is REQUIRED)
  if (!controlTimer.begin(TIMER_MODE_PERIODIC, timer_type, tindex,
     freq_hz, 0.0f, timer_cb_adapter))
    return;
  if (!controlTimer.setup_overflow_irq())
    return;
  if (!controlTimer.open())
    return;
  controlTimer.start();
}
```